

분류 ② KNN (K-Nearest Neighbors)

핵심 질문: "가까운 애들끼리 같은 라벨"은 어떻게 동작하는가?

오늘의 목표

1. **KNN 원리:** 거리 기반 다수결 분류
2. **K 값의 영향:** K가 작으면 과적합, K가 크면 과소적합
3. **스케일링의 중요성:** 단위가 다르면 거리 계산이 왜곡됨
4. **Decision Boundary 시각화:** K에 따라 경계가 어떻게 변하는지
5. **Pipeline + GridSearchCV:** 최적의 K 찾기

예제: Iris petal 2D (Decision Boundary), 스케일링 전후 비교

로지스틱 회귀 vs KNN

로지스틱 회귀는 **확률**로 분류

이번에는 완전히 다른 방식 — **거리**로 분류하는 KNN을 배울거임

비교 로지스틱 회귀 KNN
----- ----- -----
분류 기준 확률 (sigmoid) 거리 (다수결)
학습 방식 가중치(w, b) 학습 학습 없음 (데이터 저장)
경계 모양 직선(선형) 복잡한 곡선
스케일링 권장 필수

> 로지스틱 회귀: "점수가 높으면 양성"

> KNN: "가까운 이웃이 양성이면 나도 양성"

Part 1. KNN 원리 — 거리 기반 다수결

KNN이란?

1. 새로운 데이터 x 가 들어오면
2. 학습 데이터에서 x 와 **가까운 K 개** 이웃을 찾고
3. 이웃 라벨의 **다수결**로 x 의 라벨을 결정

예시 ($K=3$)

、

새 데이터 ★의 이웃 3개:

- 빨강 (거리=1.2)
- 빨강 (거리=1.5)
- 파랑 (거리=2.0)

다수결: 빨강 2 > 파랑 1 → ★ = 빨강!

、

하이퍼파라미터: K ($n_neighbors$)

| K 값 | 특징 | 문제 |

|-----|-----|-----|

| K 작음 (1~3) | 경계가 복잡 | 노이즈에 민감 → **과적합** |

| K 큼 (15~) | 경계가 단순 | 세부 패턴 놓침 → **과소적합** |

| K 적당 | 균형 | **교차검증**으로 찾기! |

> K 를 찾는 건 결국 **교차검증(CV)**으로!

거리(Distance) 계산

KNN은 기본적으로 유클리드 거리(피타고라스)를 사용

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

> 거리를 재기 때문에, 단위/범위가 다르면 큰 값이 거리를 지배

> 그래서 KNN에서는 스케일링이 필수!

Part 2. Iris KNN 기본 사용법

- 참고) iris는 교육용 데이터셋이라 분리가 쉬움

```
In [1]: import numpy as np, pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

model = KNeighborsClassifier(
    n_neighbors=5,
    metric="minkowski",
    p=2 # p=2 → 유클리드 거리
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f"[Test Accuracy] {accuracy_score(y_test, y_pred):.3f}")
print()
print(classification_report(y_test, y_pred,
                           target_names=iris.target_names,
                           digits=3))
```

[Test Accuracy] 1.000

	precision	recall	f1-score	support
setosa	1.000	1.000	1.000	10
versicolor	1.000	1.000	1.000	10
virginica	1.000	1.000	1.000	10
accuracy			1.000	30
macro avg	1.000	1.000	1.000	30
weighted avg	1.000	1.000	1.000	30

Confusion Matrix — 어떤 클래스를 틀렸는지 확인

3회차에서 배운 혼동행렬을 다시 활용해보기.

정확도만 보면 안 보이는 "어떤 클래스끼리 헛갈리는지"를 확인할 수 있음.

```
In [2]: from sklearn.metrics import confusion_matrix
import plotly.figure_factory as ff

cm = confusion_matrix(y_test, y_pred)

fig = ff.create_annotated_heatmap(
    z=cm,
    x=list(iris.target_names),
    y=list(iris.target_names),
    colorscale="Blues", showscale=True
)
fig.update_layout(
    title="Confusion Matrix (KNN, K=5)",
    xaxis=dict(title="Predicted"),
    yaxis=dict(title="Actual"),
    template="plotly_dark"
)
fig.show()
```

> Iris 데이터는 단순해서 스케일링 없이도 잘 되지만,
 > 실제 데이터에서는 반드시 스케일링을 해야 함

Part 3. 스케일링의 중요성

왜 KNN에서 스케일링이 필수인가?

KNN은 거리를 기반으로 판단함
 만약 두 특성의 단위/범위가 다르면?

특성 범위 거리 기여도
----- ----- :-----:
feature1 0 ~ 1 작음
feature2 0 ~ 1000 엄청 큼

> feature2가 거리를 지배 → feature1은 무시됨
 > 스케일링으로 범위를 맞춰야 공정한 거리 계산 가능!

```
In [3]: from sklearn.preprocessing import StandardScaler

rng_scale = np.random.RandomState(42)
X_demo = np.c_[rng_scale.rand(200), rng_scale.rand(200) * 1000]
y_demo = (X_demo[:, 0] + X_demo[:, 1] / 1000 > 1.0).astype(int)

Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(
    X_demo, y_demo, test_size=0.3, stratify=y_demo,
    random_state=42
)

acc_raw = accuracy_score(
    yd_te,
    KNeighborsClassifier(5).fit(Xd_tr, yd_tr).predict(Xd_te)
)

sc = StandardScaler()
Xd_tr_s = sc.fit_transform(Xd_tr)
Xd_te_s = sc.transform(Xd_te)
acc_scaled = accuracy_score(
    yd_te,
    KNeighborsClassifier(5).fit(Xd_tr_s, yd_tr).predict(Xd_te_s)
)

pd.DataFrame({
    "설정": ["스케일링 없음", "스케일링(StandardScaler)"],
    "Accuracy": [acc_raw, acc_scaled]
})
```

Out[3]:

	설정	Accuracy
0	스케일링 없음	0.650000
1	스케일링(StandardScaler)	0.983333

- > 스케일링 전후 Accuracy 차이가 극적임!
- > KNN에서 스케일링을 빼먹으면 성능이 폭락.

안전하게 스케일링하는 법: Pipeline

3회차에서 배운 Pipeline을 사용하면 데이터 누수 없이 스케일링 + 모델을 묶을 수 있음

```
`python
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])
```

```
In [4]: from sklearn.pipeline import Pipeline

pipe_knn = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])

knn_raw = KNeighborsClassifier(n_neighbors=5)
acc_raw = accuracy_score(y_test, knn_raw.fit(X_train,
y_train).predict(X_test))

acc_pipe = accuracy_score(y_test,
pipe_knn.fit(X_train,y_train).predict(X_test))

pd.DataFrame({
    "Model": ["KNN (스케일링 없음)", "Pipeline (Scaler + KNN)"],
    "Accuracy": [acc_raw, acc_pipe]
})
```

Out[4]:

	Model	Accuracy
0	KNN (스케일링 없음)	1.000000
1	Pipeline (Scaler + KNN)	0.933333

Q. 스케일링을 하니깐 오히려 성능이 떨어졌는데, 그럼 스케일링 안 하는 게 낫지 않나요?

핵심 답변: 스케일링이 틀린 게 아니라, 모델의 판단 기준이 바뀐 거임

위 Iris 결과를 보면 스케일링 없이 1.0, 스케일링 후 0.933이 나옴.
"스케일링이 성능을 떨어뜨렸다"고 생각하기 쉽지만, **정반대**

스케일링 전: 범위가 큰 피처가 거리 계산을 지배.

- 모델이 사실상 **그 피처 하나만 보고** 분류한 것
- 우연히 그 피처가 분류에 유용했다면 성능이 높게 나올 수 있음

스케일링 후: 모든 피처를 **동일한 척도**로 비교함

- 모델이 **모든 피처를 공정하게** 보고 분류
- 판단 기준이 바뀌었기 때문에 성능 수치가 달라질 수 있음

![스케일링 후: 모든 피처의 민주주의](스크린샷%202026-02-16%20오후%201.44.44.png)

KNN은 어떤 모델인가?

![KNN의 정체성: 공정한 재판관](스크린샷%202026-02-16%20오후%201.45.13.png)

> KNN = 공정한 거리 비교 모델

- KNN은 특정 피처에 **가중치**를 주는 모델이 아닙니다
- 내가 입력한 피처들을 **동일하게 보고, 동일한 기준(거리)**으로 비교하는 모델임
- 스케일링은 그 **"동일한 비교"**를 가능하게 만들어주는 과정

만약 피처 중요도를 학습하고 싶다면?

- 그건 **다른 모델**(로지스틱 회귀, 트리 모델 등)의 역할

그렇다면 KNN은 언제 쓰는 게 좋을까?

![그렇다면 KNN은 언제 써야 할까요?](스크린샷%202026-02-16%20오후%201.49.44.png)

Part 4. K 값에 따른 성능 비교

```
In [5]: import plotly.graph_objects as go

k_list = [1, 3, 5, 7, 9, 11, 13, 15]
acc_raw_list, acc_std_list = [], []

for k in k_list:
    acc_raw_list.append(
        accuracy_score(y_test,
                        KNeighborsClassifier(k).fit(X_train,
y_train).predict(X_test))
    )
    pipe = Pipeline([("scaler", StandardScaler()),
                      ("knn", KNeighborsClassifier(k))])
    acc_std_list.append(
        accuracy_score(y_test, pipe.fit(X_train,
y_train).predict(X_test))
    )

fig = go.Figure()
fig.add_trace(go.Scatter(x=k_list, y=acc_raw_list,
mode="lines+markers",
                        name="KNN (스케일링 없음)",
                        line=dict(color="grey", dash="dash"))))
fig.add_trace(go.Scatter(x=k_list, y=acc_std_list,
mode="lines+markers",
                        name="Pipeline (Scaler+KNN)",
                        line=dict(color="cyan"))))
fig.update_layout(title="K 변화에 따른 Accuracy (Iris)",
                  xaxis_title="K (이웃 수)",
                  yaxis_title="Accuracy",
                  template="plotly_dark",
                  yaxis=dict(range=[0.8, 1.02]))

fig.show()
```

K 값 해석

- **K=1**: 가장 가까운 1개만 보고 판단 → 노이즈에 휘둘림 (과적합)
- **K=15**: 15개 이웃의 평균 → 경계가 단순해짐 (과소적합)
- **적당한 K**: 교차검증으로 찾자!

Part 5. Decision Boundary 시각화

K 값에 따라 **분류 경계(Decision Boundary)**가 어떻게 변하는지 직접 봅시다.

Iris 데이터에서 petal length, petal width 2개 특성만 사용합니다.

- 이걸 가지고 과소적합, 과적합 확인해봅세

```

In [6]: import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
import plotly.graph_objects as go

iris = load_iris()
X_2d = iris.data[:, 2:4]
y_2d = iris.target

X_tr, X_te, y_tr, y_te = train_test_split(
    X_2d, y_2d, test_size=0.3, stratify=y_2d, random_state=42
)

pad = 0.5
xx, yy = np.meshgrid(
    np.linspace(X_2d[:, 0].min() - pad, X_2d[:, 0].max() + pad,
400),
    np.linspace(X_2d[:, 1].min() - pad, X_2d[:, 1].max() + pad,
400)
)
grid = np.c_[xx.ravel(), yy.ravel()]

k_list = [1, 3, 5, 7, 9, 15]
Z_map = {}
acc_map = {}

for k in k_list:
    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    pipe.fit(X_tr, y_tr)
    Z_map[k] = pipe.predict(grid).reshape(xx.shape)
    acc_map[k] = pipe.score(X_te, y_te)

k0 = 5
fig = go.Figure()

fig.add_trace(
    go.Contour(
        x=xx[0], y=yy[:, 0], z=Z_map[k0],
        showscale=False, opacity=0.35,
        contours_coloring="heatmap",
        hoverinfo="skip", name="Decision Boundary"
    )
)

fig.add_trace(
    go.Scatter(
        x=X_tr[:, 0], y=X_tr[:, 1], mode="markers",
        marker=dict(symbol="circle", size=9,
            color=y_tr, line=dict(width=1,
color="white")),
        name="train"
    )
)

```

```

    )
)

fig.add_trace(
    go.Scatter(
        x=X_te[:, 0], y=X_te[:, 1], mode="markers",
        marker=dict(symbol="triangle-up", size=11,
                    color=y_te, line=dict(width=1)),
        name="test"
    )
)

fig.update_layout(
    title=f"KNN Decision Boundary (K={k0}) – Test Acc: {acc_map[k0]:.3f}",
    xaxis_title="Petal length (cm)",
    yaxis_title="Petal width (cm)",
    template="plotly_dark",
    legend=dict(orientation="h", yanchor="bottom", y=1.02,
xanchor="right", x=1)
)

fig.update_layout(updatemenus=[
    dict(
        type="dropdown",
        x=1.0, xanchor="right", y=1.1, yanchor="top",
        buttons=[
            dict(
                label=f"K={k}",
                method="update",
                args=[
                    {"z": [Z_map[k]]},
                    {"title": f"KNN Decision Boundary (K={k}) –
Test Acc: {acc_map[k]:.3f}"}
                ]
            )
            for k in k_list
        ]
    )
])

fig.show()

```

Decision Boundary 해석

- **K=1**: 경계가 울퉁불퉁 → 노이즈 하나하나에 반응 (과적합)
- **K=15**: 경계가 매끈하고 단순 → 세부 패턴을 놓칠 수 있음 (과소적합)
- **K=5**: 적당한 균형

> 드롭다운을 바꿔가며 K에 따른 경계 변화를 직접 확인해보셈

Q. 중요한 피처만 넣으면 성능이 더 좋아지지 않을까?

- 위 Decision Boundary를 보면, petal length와 petal width 2개만으로도 거의 선형 분리가 됨.
- 그렇다면 이 2개만 쓰는 게 4개 전부 쓰는 것보다 낫지 않을까?

직접 비교해 봅세.

```
In [7]: from sklearn.model_selection import cross_val_score
from sklearn.model_selection import StratifiedKFold

iris = load_iris()
X_all = iris.data
X_petal = iris.data[:, 2:4]
y = iris.target

pipe_all = Pipeline([("scaler", StandardScaler()),
                      ("knn",
                       KNeighborsClassifier(n_neighbors=5))])
pipe_petal = Pipeline([("scaler", StandardScaler()),
                        ("knn",
                         KNeighborsClassifier(n_neighbors=5))])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores_all = cross_val_score(pipe_all, X_all, y, cv=cv,
                              scoring="accuracy")
scores_petal = cross_val_score(pipe_petal, X_petal, y, cv=cv,
                                scoring="accuracy")

pd.DataFrame({
    "피처 구성": ["petal 2개 (length, width)", "전체 4개 피처"],
    "사용 피처 수": [2, 4],
    "CV 평균 Accuracy": [scores_petal.mean(), scores_all.mean()],
    "CV 표준편차": [scores_petal.std(), scores_all.std()]
})
```

Out[7]:

	피처 구성	사용 피처 수	CV 평균 Accuracy	CV 표준편차
0	petal 2개 (length, width)	2	0.960000	0.038873
1	전체 4개 피처	4	0.973333	0.024944

결과 해석: "중요한 피처만 남기면 더 좋아진다"는 착각

petal 2개만 썼을 때 오히려 **4개** 전부 쓴 것보다 성능이 낮음

왜?

> "중요한 피처만 남기면 더 좋아질 것 같다"

> → 이건 **가중치 학습 모델의 사고 방식**임!

![모델마다 생각하는 방식이 다릅니다](스크린샷%202026-02-16%20오후%201.49.06.png)

| 사고 방식 | 해당 모델 | KNN에 적용? |

|-----|-----|:-----|

| 피처별 중요도를 학습 | 로지스틱 회귀, 트리 모델 | X |

| 모든 피처를 동일하게 거리 계산 | **KNN** | O |

- KNN은 **가중치를 학습하지 않음**

- 그냥 모든 피처를 다 더해서 거리를 계산

- 그래서 피처를 줄이면 → **정보량이 감소** → 성능 하락

> **KNN에서 피처를 줄이는 건 정보를 줄이는 것일 수도, 노이즈를 제거하는 것일 수도 있음**

Part 6. GridSearchCV로 최적의 K 찾기

교차검증 + 그리드 탐색

K를 사람이 수동으로 하나씩 비교하지말고, **GridSearchCV**가 자동으로 최적의 K를 찾아줌

```
`python
param_grid = {
    "knn__n_neighbors": [1, 3, 5, 7, 9, 11],
    "knn__weights": ["uniform", "distance"],
}
```

- 각 K에 대해 **5-Fold 교차검증** 수행
- 평균 정확도가 가장 높은 K를 선택
- refit=True 이면 최적의 K로 **전체 데이터에 재학습**

```
In [8]: from sklearn.model_selection import GridSearchCV, StratifiedKFold

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

param_grid = {
    "knn__n_neighbors": [1, 3, 5, 7, 9, 11],
    "knn__weights": ["uniform", "distance"],
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

gs = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring="accuracy",
    cv=cv,
    n_jobs=-1,
    refit=True,
    return_train_score=True
)

iris = load_iris()
gs.fit(iris.data, iris.target)

print(f"최적 K: {gs.best_params_}")
print(f"최적 CV Accuracy: {gs.best_score_:.4f}")
```

```
최적 K: {'knn__n_neighbors': 5, 'knn__weights': 'uniform'}
최적 CV Accuracy: 0.9733
```



```

In [9]: import pandas as pd

cv_df = pd.DataFrame(gs.cv_results_)
cv_table = cv_df[[
    "param_knn_n_neighbors",
    "mean_train_score", "std_train_score",
    "mean_test_score", "std_test_score", "rank_test_score"
]].sort_values("rank_test_score")

cv_table.rename(columns={
    "param_knn_n_neighbors": "K",
    "mean_test_score": "cv_mean_acc",
    "std_test_score": "cv_std_acc"
}, inplace=True)

cv_table

```

Out[9]:

	K	mean_train_score	std_train_score	cv_mean_acc	cv_st
4	5	0.963333	0.014530	0.973333	0.024
5	5	1.000000	0.000000	0.973333	0.024
11	11	1.000000	0.000000	0.966667	0.029
7	7	1.000000	0.000000	0.966667	0.036
6	7	0.961667	0.008498	0.960000	0.032
9	9	1.000000	0.000000	0.960000	0.038
10	11	0.958333	0.013944	0.960000	0.032
2	3	0.960000	0.016159	0.946667	0.054
3	3	1.000000	0.000000	0.946667	0.054
8	9	0.960000	0.008165	0.946667	0.033
0	1	1.000000	0.000000	0.933333	0.059
1	1	1.000000	0.000000	0.933333	0.059

```

In [10]: import plotly.graph_objects as go

cv_plot=cv_table.sort_values("K")

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=cv_plot["K"],
    y=cv_plot["cv_mean_acc"],
    mode="lines+markers",
    name="CV mean acc",
    line=dict(color="cyan")
))

fig.add_trace(go.Scatter(
    x=list(cv_plot["K"]) + list(cv_plot["K"][:, :-1]),
    y=list(cv_plot["cv_mean_acc"] + cv_plot["cv_std_acc"]) +
      list((cv_plot["cv_mean_acc"] - cv_plot["cv_std_acc"])
[:, :-1]),
    fill="toself",
    opacity=0.2, name="±1 std",
    line=dict(color="cyan")
))

fig.update_layout(
    title="KNN GridSearchCV 결과 (mean ± std)",
    xaxis_title="K (이웃 수)",
    yaxis_title="CV Accuracy",
    template="plotly_dark"
)
fig.show()

```

GridSearchCV 결과 해석

- cv_mean_acc : 5-Fold 교차검증 평균 정확도
- cv_std_acc : 분산 (작을수록 안정적)
- rank_test_score : 순위 (1이 최고)

> **K=1**(과적합)은 훈련 점수는 높지만 CV 점수는 낮을 수 있음
 > 최적의 K = 교차검증 평균이 가장 높은 것!

Part 7. 차원의 저주 — KNN은 왜 고차원에서 약해질까?

Q. 4개 피처에서는 2개보다 성능이 좋았는데, 피처를 계속 늘리면 어떻게 될까?

![피처가 많을수록 정보가 많으니 무조건 좋을까요?](스크린샷%202026-02-16%20오후%201.50.08.png)

아까 피처 4개 > 피처 2개였으니, 계속 늘리면 더 좋아지지 않을까?

> 핵심 답변: 적당한 차원 증가 = 정보 증가, 과도한 차원 증가 = 공간 붕괴

2차원에서의 "가까움"

2차원에서는 직관이 잘 작동함:

- 가까운 점은 **확실히 가깝고**
- 먼 점은 **확실히 멀다**

、

● ← 가까움 (거리 = 1.2)

★

○ ← 먼 (거리 = 8.5)

、

고차원에서의 "가까움" 붕괴

![차원의 저주: 공간이 붕괴된다](스크린샷%202026-02-16%20오후%201.50.22.png)

하지만 차원이 늘어나면(피처 수가 많아지면):

- 모든 점들이 **비슷한 거리로 퍼짐**
- 가장 가까운 점과 가장 먼 점의 **거리 차이가 줄어듦**

![모든 것이 멀어지는 세계](스크린샷%202026-02-16%20오후%201.50.37.png)

$$\frac{\text{가장 가까운 거리}}{\text{가장 먼 거리}} \rightarrow 1$$

1\$\$

> 가까운 것도 멀고, 먼 것도 멀다 → 결국 다 멀다!

> 이것을 차원의 저주(Curse of Dimensionality)라고 함.

왜 KNN에 치명적인가?

| KNN의 핵심 | 고차원에서 벌어지는 일 |

|-----|-----|

| 거리가 가까운 이웃의 **다수결**로 판단 | 모든 점이 거의 같은 거리 |

| **이웃** 개념이 핵심 | 이웃 개념이 **무의미**해짐 |

| 지역적(local) 판단 모델 | **노이즈에 매우 민감**해짐 |

→ 그래서 KNN은 고차원에서 **급격하게 성능이 떨어짐**

```
In [11]: import numpy as np
from scipy.spatial.distance import cdist
import plotly.graph_objects as go

rng_cd = np.random.RandomState(42)
dims = [2, 10, 50, 200, 500]
ratios = []

for d in dims:
    X = rng_cd.rand(200, d)
    D = cdist(X, X)

    np.fill_diagonal(D, np.inf)
    min_dist = D.min(axis=1)

    np.fill_diagonal(D, 0)
    max_dist = D.max(axis=1)

    ratios.append((min_dist / max_dist).mean())

fig = go.Figure()
fig.add_trace(go.Scatter(x=dims, y=ratios, mode="lines+markers"))
fig.update_layout(
    title="차원이 커질수록 가까운 거리 / 먼 거리 → 1",
    xaxis_title="차원 수",
    yaxis_title="min / max 거리 비율",
    template="plotly_dark"
)
fig.show()
```

차원의 저주 정리

| 차원 증가 정도 | 효과 | 이유 |

|:-----:|:-----:|:-----:|

| 적당히 (2→4→5) | 성능 **향상** | 정보가 늘어남 |

| 과도하게 (5→50→100) | 성능 **급락** | 거리의 구별력이 무너짐 |

> 적당한 차원 증가 = 정보 증가

> 과도한 차원 증가 = 거리 구별력 붕괴 (차원의 저주)

고차원 데이터를 다루려면?

KNN은 고차원에서 구조적으로 약하기 때문에, 고차원 데이터에는 다른 전략이 필요함:

- 차원 축소 (PCA 등)
- **Feature Selection** (유용한 피처만 선별)
- 다른 모델 사용 (로지스틱 회귀, 트리 모델, SVM 등)

> 다음 시간에 배울 **트리 모델(Decision Tree)**은 왜 고차원에서 강한지도 비교해 보겠음!

![요약: KNN 마스터를 위한 3가지 핵심](스크린샷%202026-02-16%20오후%201.51.23.png)

오늘의 정리

개념	핵심	코드
KNN	거리 기반, 이웃 K개 다수결	`KNeighborsClassifier(n_neighbors=K)`
스케일링	KNN에서 필수 (거리 왜곡 방지)	`StandardScaler()`
Pipeline	스케일링 + 모델을 안전하게 묶기	`Pipeline([("scaler", ...), ("knn", ...)])`
K 탐색	교차검증으로 최적 K 찾기	`GridSearchCV(pipe, param_grid, cv=cv)`
Decision Boundary	K에 따라 경계 복잡도 변화	K 작음=과적합, K 큼=과소적합
차원의 저주	고차원에서 거리 구별력 붕괴	적당한 차원↑=정보↑, 과도한 차원↑=성능↓

오늘의 핵심 Q&A 요약(지난 기수)

질문	핵심 답변
스케일링하면 성능이 떨어지는데?	모델의 판단 기준이 바뀐 것. 스케일링이 "공정한 비교"를 만드는 것
KNN은 언제 쓰는 게 좋은가?	모든 피처를 동일한 척도로 비교하여 유사성 기반 분류할 때
중요한 피처만 남기면 더 좋지 않나?	KNN은 가중치 학습 모델이 아님. 피처 줄이면 정보가 줄어들음
왜 고차원에서 약해지나?	차원의 저주: 모든 점이 비슷한 거리 → 이웃 개념이 무의미

로지스틱 회귀 vs KNN 최종 비교

| 비교 | 로지스틱 회귀 | KNN |
 |-----|-----|
 | 분류 기준 | 확률 (sigmoid) | 거리 (다수결) |
 | 학습 | 가중치 학습 (빠름) | 학습 없음 (예측 시 계산) |
 | 피처 중요도 | 학습함 (계수 = 중요도) | 학습 안 함 (모두 동일하게 취급) |
 | 경계 | 직선 (선형) | 복잡한 곡선 |
 | 스케일링 | 권장 | 필수 |
 | 고차원 | 상대적으로 강함 | 차원의 저주에 취약 |
 | 장점 | 확률 출력, 해석 쉬움 | 구현 쉬움, 비선형 경계 |
 | 단점 | 비선형 패턴 약함 | 데이터 많으면 느림, 고차원 취약 |
 | 핵심 하이퍼파라미터 | C (규제 강도) | K (이웃 수) |

> 기억할 문장:

- > 로지스틱 = "확률로 분류", KNN = "가까운 애들끼리 다수결로 분류"
- > KNN은 공정한 거리 비교 모델 — 특정 피처에 힘을 실어주지 않는다!
- > 둘 다 스케일링 + Pipeline + 교차검증이 기본!

다음 시간 예고

- 결정 트리 (Decision Tree): 질문을 반복해서 분류하는 규칙 기반 모델
- 앙상블 (Ensemble): 여러 모델을 합쳐서 더 좋은 성능 만들기
- Bias-Variance Tradeoff: 과적합/과소적합의 근본 원리

- > 오늘 확률(로지스틱)과 거리(KNN) 기반 분류를 배웠으니,
- > 다음엔 규칙 기반 분류인 결정 트리를 배울거임!
- >
- > 트리 모델은 KNN과 달리 피처 중요도를 학습하고,
- > 고차원에서도 강한 이유를 함께 비교해볼거임!
- >
- > 숙제 파일(4회차_숙제.ipynb)을 확인하고 생각해오셈!

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).